DISEÑO AUTOMÁTICO DE SISTEMAS FIABLES (VHDL)

DEBOUNCER DE LA SEÑAL DE UN RELOJ

Álvaro Terron

Gonzalo Vázquez

José María Fernández

1. INTRODUCIÓN. PRACTICA 1

En esta primera práctica se pida implementar un debouncer para la señal de entrada de un botón, a fin de controlar un LED de salida. El debouncer se utiliza para evitar pulsaciones falsas o ruido que puedan ocurrir al presionar el botón.

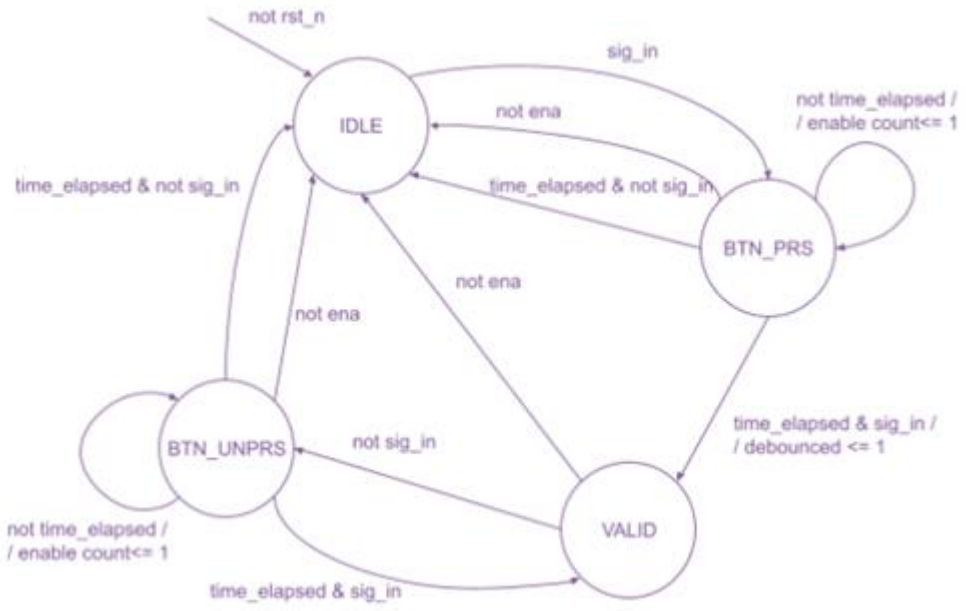
Para lograr esto, se implementará un contador que cuenta hasta determinado valor (ms). Si la señal del botón permanece estable durante este tiempo, dicha pulsación se considerará válida, generando un pulso de un ciclo de reloj en la salida del debouncer.

El sistema se compone de tres archivos VHDL: `top_practica1`, `debouncer` y `synchronizer`. Además, se proporcionan dos testbenches para probar el diseño, uno para el debouncer por separado y otro para el sistema completo.

En el archivo `top_practica1`, se deben completar las definiciones y las instancias de los bloques del proyecto.

El archivo `debouncer.vhd` es donde se implementará el debouncer. Este archivo debe ser modificado en tres procesos para implementar la funcionalidad del sistema, incluyendo el 'timer', el registro de la máquina de estados finitos (FSM) y la lógica interna de la misma.

Respecto a esto último, podremos implementar la siguiente máquina para la representación del debouncer:



Finalmente será importante considerar casos de prueba que demuestren el correcto funcionamiento del debouncer.

2. OBJETIVOS DE LA PRÁCTICA

- Desarrollar un **debouncer** para la señal de entrada de un botón, para rechazar posibles pulsaciones falsas o ruido.

- Establecer una manera para controlar el estado de un **LED** de salida.

- Implementar las definiciones y las instancias necesarias en el archivo **top_practica1**

- Realizar las adecuaciones requeridas en el archivo **debouncer.vhd**

- Utilizar los **testbenches** proporcionados para verificar el correcto funcionamiento del sistema completo

- Considerar **casos de prueba** específicos que pongan a prueba el comportamiento del debouncer en diversas situaciones.

3. IMPLEMENTACIONES

3.1. Top_practica1.vhd

Los cambios realizados en este archivo buscan el correcto funcionamiento del sistema. Estos cambios se han implementado mediante dos procesos principales en el código: `registerLED` y `toggleLED`.

Reset del sistema de forma asíncrona de acuerdo con la señal `rst_n`

Se ha agregado una condición para el reset asíncrono "`rst_n`", donde se establece el valor inicial del registro del LED como '`0`' cuando el reset está activo.

```
registerLED: process(clk100Mhz, rst_n) is
begin
  if (rst_n = '0') then
    -- ---------------------------------------------------------------------------
    -- Completar con el valor inicial del registro
    -- ---------------------------------------------------------------------------
  LED_register <= '0';
  elsif rising_edge(clk100Mhz) then
    -- ---------------------------------------------------------------------------
    -- Registrar valor
    -- ---------------------------------------------------------------------------
  LED_register <= state_LED;
  end if;
end process;
```

Aumento de un contador cuando sea necesario (síncrono)

Se ha implementado un proceso para aumentar el contador de forma síncrona con el flanco de subida (`clk`). Este permitirá contar los ciclos necesarios y determinar cuándo se ha completado el tiempo definido para el debouncer.

En caso de haber contado los ciclos necesarios, generar una señal que lo indique

Se ha definido la lógica generar una señal que indique cuando se han contado los ciclos necesarios. Esta señal se utiliza para indicar que se ha completado el tiempo de debouncer y que la señal del botón es válida.

```
-- PROCESS to toggle LED
toggleLED: process(LED_register, Toggle_LED)
begin
-- ---------------------------------------------------------------------------
  state_LED <= LED_register XOR Toggle_LED; --Completar con la lÃ³gica del LED (toggle)
-- ---------------------------------------------------------------------------
end process;
-- Connect LED_register to the output
LED <= LED_register;
```

Estos cambios permiten asegurar que el sistema funcione correctamente en términos de registro del estado del LED de salida, conteo de ciclos y generación de señales indicativas de estado. Con estas modificaciones, se completa la funcionalidad del archivo top_practica1.vhd para integrar adecuadamente el debouncer con el LED de salida y el sistema en general.

3.2. Debouncer.vhd

Estados de nuestra máquina (FSM)

Debemos declarar los distintos estados que pueden pasar en nuestra FSM (`IDLE`, `BTN_PRS`, `VALID` y `BTN_UNPRS`) y dos señales que representan el estado actual y el estado siguiente (`current_state`, `next_state`)

Además, una señal `count` actuará como contador de ciclos desde la última vez que se activó la salida del debouncer. También hemos definido dos señales auxiliares: `time_elapsed` y `debounced_aux`, una para comprobar cuándo saltó por última vez nuestro debouncer y otra para poder asignar nuestra señal de salida final.

```
-- ---------------------------------------------------------------------------
-- Declarar un tipo para los estados de la fsm usando type
-- ---------------------------------------------------------------------------

type state is (IDLE, BTN_PRS, VALID, BTN_UNPRS);
signal current_state, next_state: state;

signal count: unsigned (c_counter_width-1 downto 0);
signal time_elapsed, debounced_aux: std_logic;
```

Implementación del Timer

Este código forma parte de un proceso cuyas señales de sensibilidad la forman una señal de reloj y una señal de reset. Este timer está diseñado para implementar el sistema antirrebote para la pulsación de un botón. El proceso trata de verificar si el botón está presionado o no, incrementando un contador o no.

Cuando el contador alcanza un cierto número de ciclos, se reinicia y una variable llamada 'time_elapsed' se establece en '1'. Esta variable es crucial para determinar la validez de la pulsación del botón.

En resumidas cuentas…

- Si el estado actual es **BTN_PRS** o **BTN_UNPRS**, significa que el botón está presionado o liberado. Así, se incrementa el contador de ciclos (**count**) mientras se cumplan las condiciones. Si el contador alcanza el número de ciclos (**c_cycles**) determinado y el siguiente estado no es **VALID**, se establece la variable **time_elapsed** en '1' para indicar que la pulsación del botón ha sido válida. Se reinicia el contador.
- Si el estado actual es **IDLE**, o el siguiente estado es **BTN_PRS** o **BTN_UNPRS** después de un estado **VALID**, también se incrementa el contador.

```
begin
    --Timer
    process (clk, rst_n)
    begin
    -- ---------------------------------------------------------------------------
    -- Completar el timer que genera la seÃƒÂ±al de time_elapsed para trancionar en
    -- las mÃƒÂ¡quinas de estados
    -- ---------------------------------------------------------------------------
        if(rst_n = '0') then
            count <= (others => '0');
        elsif (rising_edge(clk)) then
            time_elapsed <= '0';
            if(current_state=BTN_PRS) then
                if ((count < c_cycles) and (next_state /= VALID))then
                    count<= count +1;
                else
                    time_elapsed <= '1';
                    count <= (others => '0');
                end if;
            elsif(current_state=BTN_UNPRS) then
                if (count < c_cycles) and (next_state /= VALID)then
                    count<= count +1;
                else
                    time_elapsed <= '1';
                    count <= (others => '0');
                end if;
            elsif((current_state=IDLE) or ((current_state=VALID) and (next_state = BTN_PRS or next_state= BTN_UNPRS ))) then
                count <= count+1;
            end if;
            end if;
    end process;
```

Registro del siguiente estado

Este proceso servirá como el registro de FSM para determinar el próximo estado del sistema. Se activa por el flanco de subida del reloj y la señal de reset asíncrono (sensibilidad)

- Si el reset está activo, el sistema se reinicia y se establece en un estado de reposo. La salida del debouncer se desactiva.

- Si no hay reset y hay un flanco de subida del reloj, se actualiza el estado actual con el siguiente, determinado por la lógica de la máquina de estados, y se refleja el estado del debouncer en la salida del sistema.

Este proceso garantiza un comportamiento sincronizado y estable.

```
--FSM Register of next state
process (clk, rst_n) begin
    if(rst_n = '0') then
        current_state <= IDLE;
        debounced <='0';
    elsif (rising_edge(clk)) then
        current_state <= next_state;
        debounced <= debounced_aux;
    end if;

end process;
```

Implementación de la FMS

```
    process (ena,sig_in, current_state,time_elapsed)
    begin
        debounced_aux<='0';
        case current_state is
            when IDLE =>
                if(sig_in = '1') then
                    next_state <= BTN_PRS;
                else
                    next_state <= current_state;
                end if;
            when VALID =>
                if (ena = '0') then
                    next_state <= IDLE;
                elsif (sig_in = '0') then
                    next_state <= BTN_UNPRS;
                else
                    next_state <= current_state;
                end if;
                when BTN_PRS =>
                if( ena = '0') then
                    next_state <= IDLE;
                elsif (time_elapsed = '0') then
                    next_state <= current_state;
                elsif ((time_elapsed = '1') and (sig_in = '0')) then
                    next_state <= IDLE;
                elsif ((sig_in = '1') and (time_elapsed = '1')) then
                    next_state <= VALID;
                    debounced_aux<='1';
                else
                    next_state <= current_state;
                end if;
            when BTN_UNPRS =>
                if ( time_elapsed = '0') then
                    next_state <= current_state;
                elsif ((ena = '0') or (time_elapsed = '1')) then
                    next_state <= IDLE;
                else
                    next_state <= current_state;
                end if;
            when others =>
                next_state <= IDLE;
        end case;
    end process;
end Behavioural;
```

Este bloque implementa la lógica de la máquina de estados finitos (FSM). Se analizan las señales de enable, señal de entrada para el debouncer, el último cambio realizado y el estado actual.

Así, dependiendo del estado actual, se determina el próximo estado de acuerdo con las condiciones especificadas:

- **IDLE**:

  - Si **sig_in** es '1', pasa al estado **BTN_PRS**
  - De lo contrario, permanece en **IDLE**.

- **VALID**:

  - Si **ena** es '0', regresa a **IDLE**;
  - si **sig_in** es '0', pasa a **BTN_UNPRS**;
  - De lo contrario, permanece en **VALID**.

- **BTN_PRS**:

  - Si **ena** es '0', regresa a **IDLE**;
  - si **time_elapsed** es '0', permanece en el mismo estado
  - si **time_elapsed** es '1' y **sig_in** es '0', regresa a **IDLE**
  - si **sig_in** y **time_elapsed** son '1', pasa a **VALID** y se establece **debounced_aux** en '1'
  - De lo contrario, permanece en **BTN_PRS**.

- **BTN_UNPRS**:

  - Si **time_elapsed** es '0', permanece en el mismo estado
  - si **ena** es '0' o **time_elapsed** es '1', regresa a **IDLE**
  - De lo contrario, permanece en **BTN_UNPRS**.

- Si se encuentra en cualquier otro estado, regresa a **IDLE**.

Esta lógica garantiza una transición adecuada entre los estados de la FSM, asegurando un funcionamiento estable y controlado del debouncer.

4. CASOS DE USO